



VNiVERSIDAD
D SALAMANCA

Facultad de Ciencias
Grado en Ingeniería Informática

TRABAJO FIN DE GRADO

Construcción de una IA para Riichi Mahjong mediante Transformers

Autor

Julia Ruiperez de Brito

Tutor

Prof. Vidal Moreno Rodilla

Departamento

Informática y Automática

Salamanca
Junio 2026

Abstract

This project presents an Artificial Intelligence system capable of selecting discard actions in Riichi Mahjong, a traditional Japanese game of imperfect information.

Using a Transformer-based architecture, the model is trained through imitation learning on a dataset composed of real games collected from Tenhou.net. The model is evaluated in terms of predictive accuracy, and the implications of the obtained results are discussed.

In addition to conventional accuracy metrics, a novel evaluation methodology, referred to as *Strategic Accuracy*, is proposed. This metric is based on semantic categories derived from Mahjong strategy and provides a deeper understanding of the quality of the model's decisions.

The experimental results show that the model achieves a strategic accuracy of approximately 96%, suggesting that it is capable of learning complex gameplay patterns and making informed decisions in situations characterized by imperfect information.

Keywords:

Riichi Mahjong, Artificial Intelligence, Transformers, Imitation Learning, Strategic Accuracy, Model Evaluation, Imperfect Information Games.

Resumen

Este trabajo presenta un sistema de Inteligencia Artificial capaz de tomar decisiones de descarte en Riichi Mahjong, un juego tradicional japonés de información imperfecta caracterizado por un elevado nivel de complejidad estratégica.

Para ello, se desarrolla un modelo basado en la arquitectura Transformer entrenado mediante aprendizaje por imitación a partir de un conjunto de datos compuesto por partidas reales obtenidas de la plataforma Tenhou.net. El rendimiento del modelo se evalúa utilizando métricas convencionales de precisión, analizando además las implicaciones de los resultados obtenidos.

Con el objetivo de complementar estas métricas tradicionales, se propone una nueva metodología de evaluación denominada *Precisión Estratégica*, basada en categorías semánticas propias del juego. Este enfoque permite analizar la calidad de las decisiones generadas por el modelo desde una perspectiva más cercana al razonamiento estratégico empleado por jugadores expertos.

Los resultados experimentales muestran que el modelo alcanza una precisión estratégica cercana al 96%, lo que sugiere que es capaz de aprender patrones de juego complejos y generar decisiones coherentes en entornos caracterizados por información imperfecta.

Palabras clave:

Riichi Mahjong, Inteligencia Artificial, Transformers, Aprendizaje por Imitación, Precisión Estratégica, Evaluación de Modelos, Juegos de Información Imperfecta.

Índice general

1	Introducción	6
2	Objetivos	6
3	Conceptos Teóricos	7
3.1	Reglas Básicas	7
3.2	Heurísticas	10
3.2.1	Ukeire	10
3.2.2	Shanten	10
3.3	Transformers	10
3.4	Aprendizaje por Imitación	12
4	Estado Del Arte	13
5	Herramientas y Entorno de Desarrollo	13
6	Desarrollo del Proyecto	13
6.1	Análisis de Exploración de Datos	13
6.2	Distribución de Acciones	14
6.3	Distribución de la Suma de Acciones Válidas de una Sample	15
6.4	Frecuencia de Piezas Descartadas	16
6.5	Representación de un Estado	16
7	Arquitectura del Modelo	18
7.1	Tokenization	20
7.1.1	Tokens de Mano	20
7.1.2	Tokens de Descarte	20
7.1.3	Tokens de Melds	21
7.1.4	Tokens de Acciones	21
7.1.5	Tokens de Estado de Juego	21
7.1.6	Tokens de Estado de Jugadores	22
7.2	Diseño del Embedding	22
7.2.1	Embedding de Tokens de Piezas	22
7.2.2	Embedding del índice de copia	23
7.2.3	Embedding de Jugador	23
7.2.4	Embeddings Posicionales	23
7.2.5	Embeddings de Metadatos	23
7.2.6	Embedding de Token de Mano	23
7.2.7	Embedding de Token de Descarte	23
7.2.8	Embedding de Token de Meld	24
7.2.9	Embedding de Token de Acción	25
7.2.10	Embedding de Token de Estado de Juego	25
7.2.11	Embedding de Token de Estado de Jugador	26
	Glosario de términos de Mahjong	27

Índice de figuras

1	Posición relativa de los jugadores y vientos en una mesa de Riichi Mahjong.	8
2	Arquitectura Estándar de un Transformer	11
3	Arquitectura BERT (izquierda) vs Arquitectura GPT	12
4	Distribución de Número de Datos del Dataset por tipo de Acciones Válidas	14
5	Distribución del número de acciones válidas del schema de Descarte	15
6	Frecuencia de las Piezas Descartadas	16
7	Arquitectura del Modelo Transformer	19

Índice de cuadros

1	Objetivos del proyecto	7
2	Categorías de piezas de Riichi Mahjong	7
3	Formas de ganar en Riichi Mahjong	9
4	Acciones posibles en cada turno de Riichi Mahjong	9
5	Schemas del Dataset	14
6	Acciones válidas según melds abiertos	15
7	Campos de un Estado de Mahjong	17
8	Campos de un Jugador en un Estado de Mahjong	18
9	Campos de un Meld en un Estado de Mahjong	18
10	Campos de una Acción Válida en un Estado de Mahjong	18
11	Categorías de Informaciones	20
12	Campos de Token de Mano	20
13	Campos de Token de Descarte	20
14	Campos de Token de Melds	21
15	Campos de Token de Acciones	21
16	Campos de Token de Estado de Juego	22
17	Campos de Token de Estado de Jugadores	22

1 Introducción

Riichi Mahjong es un juego de información imperfecta, caracterizado por una elevada complejidad estratégica e información oculta, que llega a 10^{48} [6] posibles siguientes estados. Además de eso, también existe un gran caos secuencial, debido a que cada jugador tiene la posibilidad de saltar turnos a otros jugadores mediante acciones como el *pon*, el *chi* o el *kan*, lo que incrementa aún más la complejidad del juego.

Este proyecto crea una Inteligencia Artificial que selecciona el mejor descarte dado un estado de juego completo, donde existe información temporal secuencial y estática. El modelo es basado en la arquitectura de Transformers, en la cual se entrena mediante aprendizaje por imitación de 500 mil estados de juego reales obtenidos de la plataforma Tenhou.net, siendo estos juegos de alto nivel, con jugadores profesionales y amateurs avanzados.

Durante el desarrollo del proyecto, obtenemos diferentes métricas de precisión, analizando la precisión global de imitación, así como la precisión estratégica, que se basa en categorías semánticas propias del juego, como *Shanten* y *Ukeire*.

El esquema general de esta memoria será empezará con los objetivos del proyecto, seguido de una sección de teoría, donde se explicará el juego de Mahjong y las heurísticas del juego, así como la arquitectura de Transformers y el aprendizaje por imitación. Después, se hará un repaso del estado del arte, donde se analizarán los trabajos relacionados con el juego de Mahjong y con el uso de Transformers en juegos de información imperfecta. A continuación, se describirán las herramientas utilizadas para el desarrollo del proyecto, y se explicará el proceso de desarrollo del proyecto, desde la preparación de los datos hasta el entrenamiento del modelo y la evaluación de los resultados. Finalmente, se presentarán los resultados obtenidos y se discutirán las conclusiones del proyecto.

Este trabajo se desarrollo utilizando metodología ágil y se documenta de forma detallada cada una de las etapas del proyecto, desde la definición de los objetivos hasta la presentación de los resultados obtenidos, proporcionando una visión completa del proceso de construcción de la IA de Mahjong mediante Transformers.

2 Objetivos

El objetivo principal de este trabajo es conseguir desarrollar una IA de Mahjong mediante la tecnología moderna de Transformers, en contraste con los enfoques de otras IAs como Suphx[6] basado en Redes Convolucionales. Presentando los beneficios y limitaciones de el uso de los Transformers para juegos de información imperfecta.

Nos planteamos los siguientes objetivos:

Cuadro 1: Objetivos del proyecto

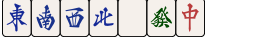
ID	Tipo	Objetivo	Prioridad
OBJ-01	General	Desarrollar un agente capaz de descartar piezas de Mahjong de forma autónoma.	Alta
OBJ-02	Funcional	Implementar un entorno de simulación de partidas de Mahjong.	Alta
OBJ-03	Funcional	Entrenar el agente mediante aprendizaje por refuerzo.	Alta
OBJ-04	Funcional	Evaluar el rendimiento del agente frente al dataset de test y heurísticas.	Alta
OBJ-05	No funcional	Garantizar que el entorno de simulación sea reproducible y configurable.	Media
OBJ-06	No funcional	Documentar el código y el proceso de entrenamiento.	Media
OBJ-07	General	Analizar el estado del arte en agentes de juegos de información imperfecta.	Baja

3 Conceptos Teóricos

En esta sección trataremos los conceptos teóricos a empezar por las reglas básicas [5] y heurísticas [1] de Riichi Mahjong, el objeto de este proyecto. Luego, pasaremos a la teoría de IA, aprendizaje por imitación y la arquitectura de *Transformers*.

3.1 Reglas Básicas

En un juego de Riichi Mahjong, contamos con 4 jugadores de los cuales 1 es el *Dealer*, representado por el viento **Este**. Cada jugador recibe 13 piezas, que pueden ser de las siguientes:

Suit	Tiles
Manzu (Caracteres)	
Pinzu (Círculos)	
Souzu (Bambús)	
Honores (Vientos y Dragones)	

Cuadro 2: Categorías de piezas de Riichi Mahjong

Las piezas de caracteres, círculos y bambús se numeran del 1 al 9, y las piezas de honores son, respectivamente, **Este**, **Sur**, **Oeste** y **Norte**; y **Dragon Blanco**, **Dragon Verde** y **Dragon Rojo**. Cada una de estas piezas tienen 4 copias, por lo que en total son 136 piezas.

Inicialmente se decide si el juego durará una ronda de mesa **Este** o una ronda hasta **Sur**, cada ronda de **Este** o de **Sur** tiene 4 juegos, donde la mesa obtiene ese viento, y aleatoriamente se escoge una persona para ser el *Dealer*, que estará representado por la pieza del **Este** y se sentará en esa posición. Cada posición de jugadores está en un viento, como podemos ver en la siguiente imagen:

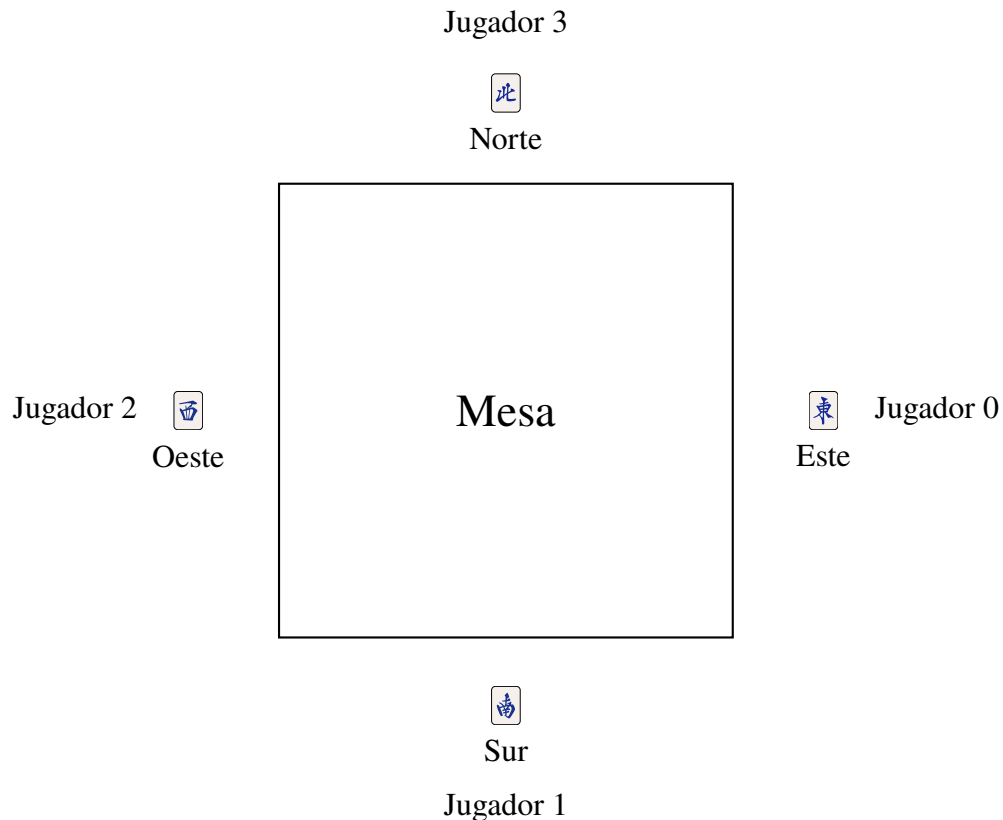


Figura 1: Posición relativa de los jugadores y vientos en una mesa de Riichi Mahjong.

En el caso de que el ganador del juego sea el Este/Dealer, la partida seguirá en el mismo estagio de ronda, por ejemplo, si estamos en el Este 3 y gana el Dealer, se hace más una ronda de Este 3.

El objetivo del juego es ganar las manos mediante *Yaku* (combinaciones de manos que pueden ganar compuestas de 4 *Melds* y un par), de forma similar al poker. Se puede ver en el Anexo 1 una lista de *Yaku* oficiales.

Cada jugador recibe 13 piezas, y su turno se basa de forma sencilla, en comprar una pieza y descartar otra, hasta que la mano se complete (si posible).

En general, una mano se caracteriza por 4 *Melds* caracterizados por 3 piezas que pueden ser un trio o una secuencia y además de esos 4 *Melds*, un par.

Al momento de completar una mano preparada para ganar (*Tenpai*), el jugador puede ganar cogiendo la pieza de la compra normal, o del descarte de otro jugador. De este modo, los jugadores también deben evitar descartar la pieza que completa la mano de otro jugador, ya que pierde puntos de este modo. En resumen, las formas de ganar son las siguientes:

Forma de Ganar	Descripción
Tsumo	Ganar con la pieza de la compra normal.
Ron	Ganar con la pieza del descarte de otro jugador.

Cuadro 3: Formas de ganar en Riichi Mahjong

El juego también dispone de una pared de piezas, donde se encuentra la llamada **Pared Muerta** (*Dead Wall*) que contiene 14 piezas que nunca se verán dentro del juego, estas 14 piezas son escogidas aleatoriamente al inicio de cada ronda. Además, esta pared muerta contiene lo que llamamos indicadores de *dora*, que son piezas que indican que la siguiente pieza de la secuencia adiciona puntos a la mano ganadora. Para los vientos, el orden de secuencia para indicar el *dora* es Este → Sur → Oeste → Norte → Este, y para los dragones es Rojo → Verde → Blanco → Rojo. Para las piezas de caracteres, círculos y bambús, el orden es 1 → 2 → 3 → ... → 9 → 1.

Durante el juego, un jugador tiene la opción de hacer "llamadas", estas llamadas se resumen a coger el descarte de otro jugador para completar un trio o una secuencia (uno de los *Melds*), abriendo tu mano y bloqueando las piezas que forman el *Meld* llamado, también se puede llamar para completar una cuadrupla, que abrirá otro indicador *dora*, pudiendo aumentar el valor de tu mano o de una mano rival. Sin embargo, hacer llamadas también tiene la contra parte de que tu mano ya no es cerrada, y por lo tanto no podrás declarar *Riichi* como veremos a continuación, una excepción es hacer un *Kan* (cuadrupla) cerrado, esto es, completar una cuadrupla con piezas de tu mano sin haberlas descartado y sin llamar a una pieza del descarte de otro jugador, este movimiento abre las 4 piezas para que los otros jugadores las vean, no obstante, si no se coge del descarte de otro jugador, tu mano sigue cerrada, teniendo la posibilidad aún de hacer *Riichi*.

La regla más importante del Riichi Mahjong: hacer ***Riichi***: Cuando un jugador está en *Tenpai* y tiene la mano cerrada, tiene la opción de declarar *Riichi*, una apuesta de 1000 puntos a que ganará la mano. Esto bloquea la mano del jugador y solo podrá comprar y descartar piezas sin poder modificar más su mano, siendo la única opción posible hacer *Tsumo* o *Ron3*.

Por lo tanto, podemos ver en la siguiente tabla un resumen de las acciones posibles en cada turno:

Acción	Descripción
Comprar	Tomar una pieza de la pared.
Descartar	Descartar una pieza de la mano.
Declarar <i>Riichi</i>	Apostar 1000 puntos a que ganará la mano.
<i>Pon</i>	Coger el descarte de otro jugador para completar un trio.
<i>Chi</i>	Coger el descarte de otro jugador para completar una secuencia.
<i>Kan</i>	Completar una cuadrupla (puede ser de tu mano sin abrir).

Cuadro 4: Acciones posibles en cada turno de Riichi Mahjong

Una última regla importante es el concepto de *furiten*, un jugador no puede ganar mediante *Ron* con una pieza que ha descartado previamente en el juego, esta regla permite el juego defensivo, ya que se sabe a todo momento con cuales piezas otros jugadores **no** pueden ganar.

Con esto finalizamos la explicación de las reglas básicas del juego, y a continuación se explicarán los conceptos de estrategia y teoría que se utilizan para jugar el juego de forma

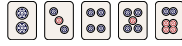
óptima.

3.2 Heurísticas

En esta subsección se definirá dos heurísticas importantes del Mahjong, el *Ukeire* y el *Shanten*, que serán utilizados más adelante como métricas de evaluación del modelo.

3.2.1 Ukeire

El *Ukeire* nada más es que el número de piezas aceptables en cada turno para mejorar tu meld, por ejemplo, para completar un **Meld** con las dos piezas , se pueden aceptar las siguientes piezas:



si ninguna de estas piezas han salido en el juego, entonces tenemos un *Ukeire* de $4 + 3 + 4 + 3 + 4 = 18$. Por lo tanto, queremos maximizar el *Ukeire* en cada turno, ya que esto nos da más posibilidades de mejorar nuestra mano y llegar a *Tenpai*.

3.2.2 Shanten

El *Shanten* se define por el número de turnos que hace mejorar la mano para llegar a *Tenpai*. Un *Shanten* de 0 indica que la mano está en *Tenpai* (falta solo una pieza para ganar), igualmente, una mano en 1-*Shanten* indica que falta una pieza para llegar a *Tenpai* y así por delante. Por lo tanto, queremos minimizar el *Shanten* en cada turno, ya que esto nos acerca a ganar la mano. El *Tenpai* también puede ser definido como el estado de la mano en el que se está a una pieza de ganar, es decir, cuando el *Shanten* es 0.

3.3 Transformers

Un Transformer es una arquitectura de Deep Learning basado en los mecanismos de *multi-head self-attention*[8], siendo variaciones ampliamente adoptado en la creación de *LLMs* (*Large Language Models*) como Chat GPT (*Generative Pre-Trained Transformer*).

La arquitectura original de un Transformer es la que podemos ver a continuación en la Figura 2, obtenida del Paper **Attention Is All You Need**[8].

Normalmente, un Transformer es compuesto por una estructura de un codificador y un decodificador. El codificador se encarga de mapear una entrada compuesta por una secuencia de símbolos a una secuencia de representaciones continuas. Por su vez, el decodificador se encarga de generar una salida compuesta de otra secuencia de símbolos. El modelo se retroalimenta de cada resultado generado más los anteriores para generar el próximo símbolo.

No obstante, un transformer encoder-decoder es computacionalmente intensivo[7], por lo que existen otras arquitecturas como **BERT** (*Bidirectional encoder representations from transformers*) que solucionan este problema al crear una arquitectura con solo el codificador[3], como podemos ver en la figura 3, utilizado en casos en el que no es necesario generar una secuencia de tokens y, además, el contexto de los tokens no se importa con unidireccionalidad de la secuencia del texto y permite un ajuste de pesos en la fase de *multi-head self-attention* de forma bidireccional. Esta arquitectura es muy útil para nuestro modelo, ya que la entrada es una serie de *Tokens* de un estado de juego del cual la posición secuencial no es importante para el significado contextual del estado de juego.

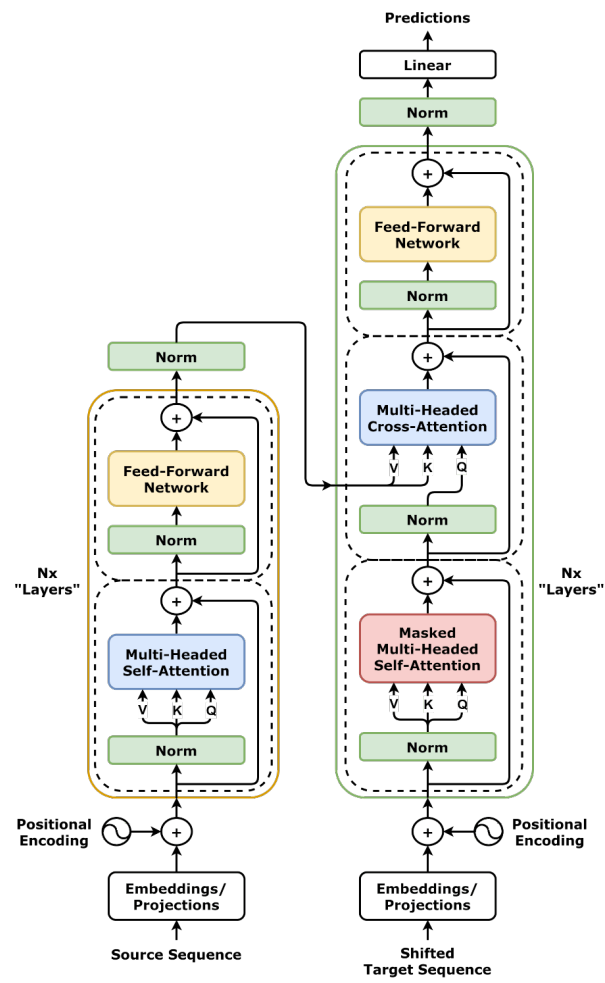


Figura 2: Arquitectura Estándar de un Transformer

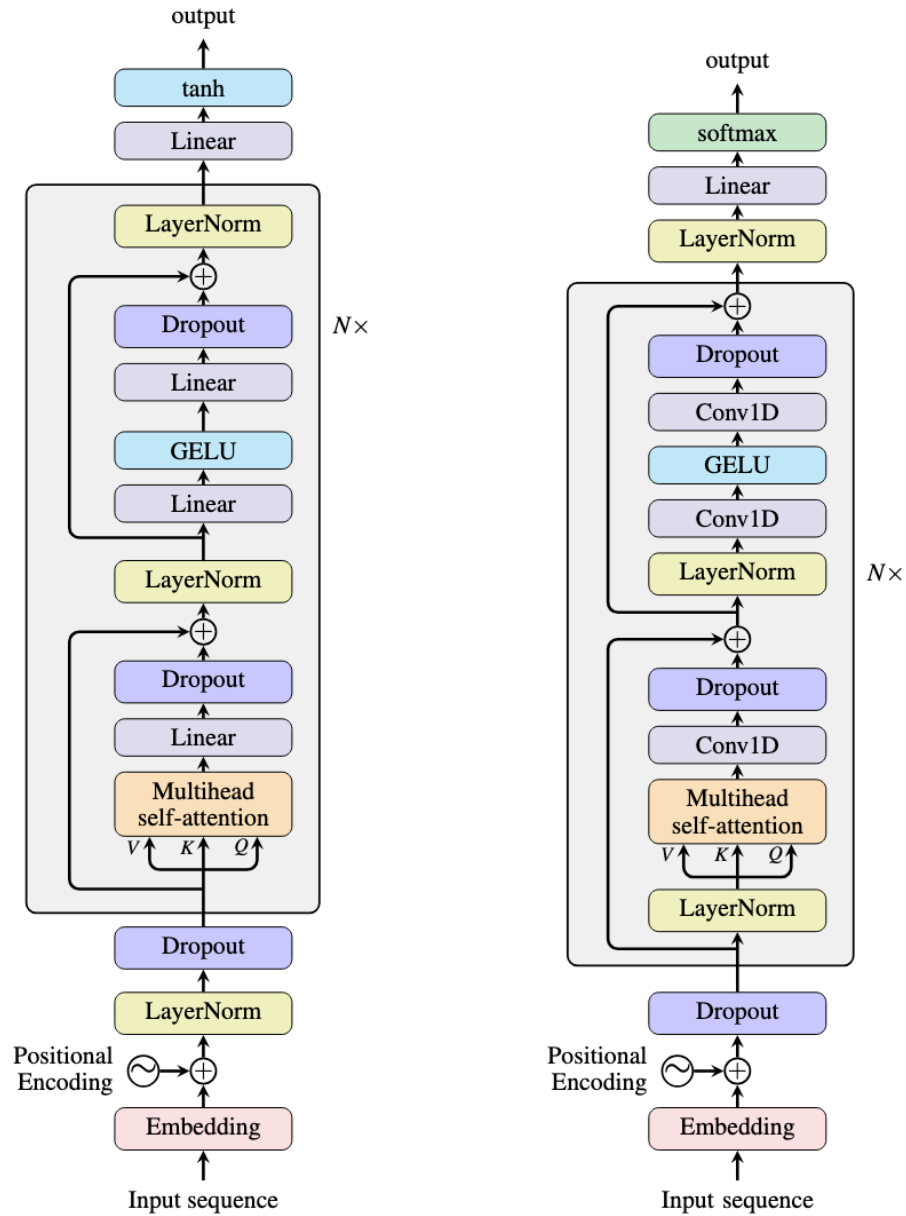


Figura 3: Arquitectura BERT (izquierda) vs Arquitectura GPT

3.4 Aprendizaje por Imitación

El aprendizaje por imitación es un tipo de aprendizaje para máquinas que se basa en copiar comportamientos humanos a partir de una cantidad de datos suficientemente grande que permita a la máquina extraer los patrones de la tarea[4].

Este tipo de aprendizaje es muy comunmente utilizado para que la máquina aprenda a jugar, debido a que se pueden conseguir muchos datos de estados de juego que permiten que una máquina consiga extraer as informaciones como reglas del juego, estrategia, decisiones estilísticas, etc.

4 Estado Del Arte

La Inteligencia Artificial capaz de jugar al Mahjong más conocida e importante es Suphx[6], una IA creada por Microsoft utilizando una arquitectura de CNNs (Redes Neuronales Convolucionales) que después pasa por una arquitectura de RL (Reinforcement Learning). Suphx alcanzó resultados muy buenos, como llegar al top 0.1% de la plataforma Tenhou.net, una de las más grandes plataformas de Riichi Mahjong Online.

No obstante, su arquitectura convolucional utiliza datos que ocupan mucha memoria espacial, de forma muy extensiva, debido a que transforma los datos de estados de juego de Mahjong en imágenes, por lo que otros modelos como el Tjong[2] buscan resolver ese problema utilizando transformers, de forma a que cada estado del juego obtiene un preprocesado mucho más sencillo y eficiente, el modelo de Tjong, no obstante, está entrenado para el Mahjong de la variación China, debido a eso, este trabajo propone un modelo de clasificación de descartes de Riichi Mahjong (la variación Japonesa de Mahjong) mediante Transformers.

5 Herramientas y Entorno de Desarrollo

El desarrollo del proyecto fue realizado en un entorno de Python con Conda y Cuda para Pytorch. Mediante iteraciones utilizando jupyter notebooks, y controlador de versiones Github en este enlace: <https://github.com/Ruipi/tfg>.

Además de eso, contamos con un ordenador con una GPU NVIDIA GeForce RTX 5060, que nos permite entrenar el modelo de Transformers de forma eficiente y rápida.

6 Desarrollo del Proyecto

En esta sección explicaremos como ha sido desarrollado el proyecto, empezando por el Análisis de Exploración de Datos (EDA); el planteamiento de la arquitectura, donde se verán las decisiones tomadas para la construcción del modelo; la construcción del modelo; y las iteraciones de desarrollo.

6.1 Análisis de Exploración de Datos

A partir del Dataset de Kaggle 4-players Riichi Mahjong Dataset se creó un notebook donde se explora todos los *schemas* del dataset.

El dataset es compuesto por 7 *schemas*:

Cuadro 5: Schemas del Dataset

ID	Nombre	Descripción
0	Skip	No realizar ninguna acción; pasar el turno.
1	Discard	Descartar una ficha de la mano.
2	Chi	Robar la ficha descartada por el jugador anterior para formar una secuencia.
3	Pon	Robar la ficha descartada por cualquier jugador para formar un trío.
4	DaiMinKan	Robar la ficha descartada por cualquier jugador para formar un cuádruple abierto.
5	ShouMinKan	Añadir una ficha propia a un trío abierto (Pon) para formar un cuádruple.
6	AnKan	Declarar un cuádruple cerrado con cuatro fichas de la propia mano.
7	Riichi	Declarar <i>Tenpai</i> con la mano cerrada, apostando 1000 puntos.

Y a partir de ellos, se analizaron los siguientes datos.

6.2 Distribución de Acciones

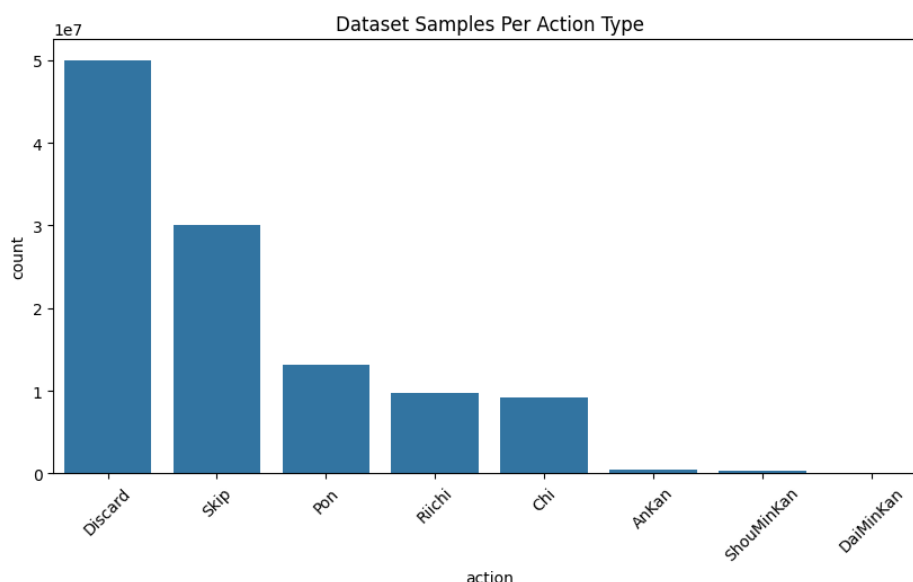


Figura 4: Distribución de Número de Datos del Dataset por tipo de Acciones Válidas

En este gráfico podemos observar un desbalance de acciones, no obstante, se observó que este desbalance es fiel a la realidad, ya que en juegos de alto nivel la cantidad de acciones que abren la mano muy raramente ocurre, y la cantidad de Kan, además de ser baja porque las probabilidades de conseguir las 4 copias de una ficha en el juego es baja, también es un movimiento de mucho riesgo, por lo que eso explica las pocas ocurrencias en la distribución. En conclusión, apesar de este desbalance que se tendrá que llevar en cuenta para los modelos a parte del descarte, en general es correcto que el modelo se entrene sobre estas cantidades inicialmente. Sin embargo, nuestro modelo actualmente solo está entrenado sobre la acción de Descartes, siendo las otras acciones una implementación futura.

6.3 Distribución de la Suma de Acciones Válidas de una Sample

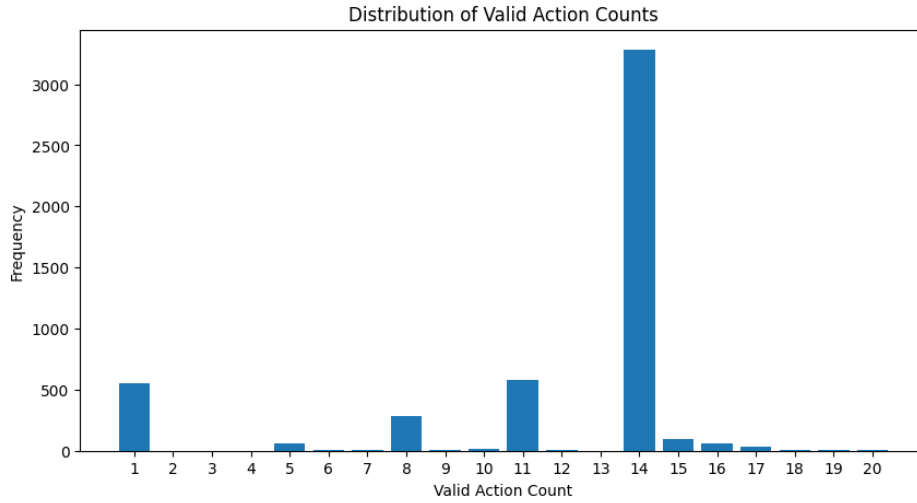


Figura 5: Distribución del número de acciones válidas del schema de Descarte

Observamos que la mayor concentración ocurre en 14 acciones validas, esto ocurre debido a que en la mayor parte del tiempo, el jugador debe descartar una pieza de 14. Luego, aparecen picos en 11, 8 y 5 acciones válidas; esto corresponde naturalmenete a estados de manos abiertas, como podemos ver en la siguiente tabla:

Cuadro 6: Acciones válidas según melds abiertos

Acciones válidas	Melds abiertos
11	1
8	2
5	3
2	4

El 2 obtiene un número bajo debido a que jugadores de alto nivel no suelen abrir la mano hasta que resten 2 piezas de acciones posibles, ya que se busca maximizar las acciones hasta llegar a *Tenpai*.

Además de esto, vemos un alto número donde solamente existe 1 acción válida. Esto se explica con *Riichi*, ya que el *Riichi* bloquea la mano del jugador, permitiendo apenas descartar la pieza comprada o ganar con ella. En función de esto, consideramos limpiar el dataset de estos casos, ya que introduce mucho ruido al modelo, considerando que el jugador ya no tiene autonomía a partir del momento en que hace *Riichi*.

Finalmente, el número de acciones válidas restante se explica debido a posibles acciones de *Riichi* con diferentes piezas.

6.4 Frecuencia de Piezas Descartadas

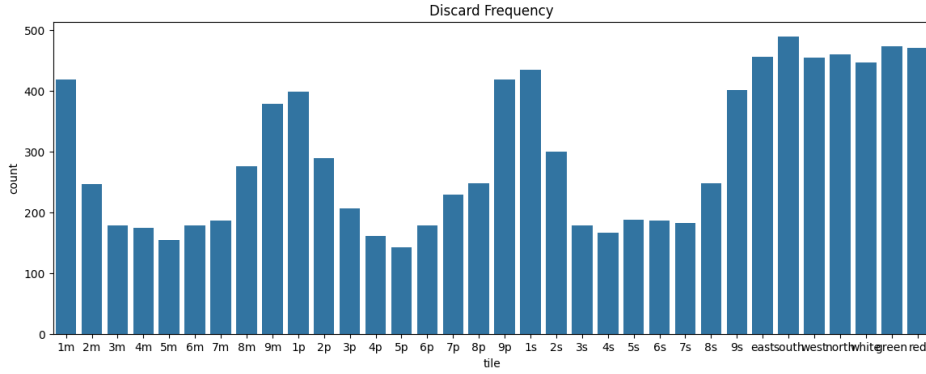


Figura 6: Frecuencia de las Piezas Descartadas

En este gráfico podemos ver que las piezas con mayor frecuencia de dedscarte son los Honores (Vientos y Dragones), ver Tabla 2, y las piezas terminales (unos y nueves), esto se debe a que estas piezas disminuyen el *Ukeire*, ver 3.2.1 Ukeire.

6.5 Representación de un Estado

Esta es la representación de un estado de mahjong dentro del Dataset:

Listing 1: Ejemplo de un estado

```
{
  { 'round_wind': 0,
    'num_honba': 1,
    'num_riichi': 0,
    'player_wind': 0,
    'position': 0,
    'remain_tiles': 15,
    'dora_indicators': [7, 5],
    'hand_tiles': [1, 1, 2, 3, 3, 12, 13, 18, 19, 22, 22],
    'players': { '0': { 'points': 41700,
                        'riichi': False,
                        'discards': [28, 27, 9, 16, 10, 6, 15, 20, 13, 32, 32, 0, 33, 33],
                        'melds': [{ 'type': 2, 'tiles': [92, 98, 103, -1], 'who': [0, 0, 3, -1]}],
                        'tsumo_giri': [0, 0, 0, 1, 0, 0, 1, 0, 0, 1, 1, 0, 0, 0]},
                  '1': { 'points': 19000,
                        'riichi': False,
                        'discards': [30, 27, 32, 18, 31, 17, 26, 26, 24, 2, 0, 14, 30, 10],
                        'melds': [],
                        'tsumo_giri': [0, 0, 0, 1, 0, 0, 0, 0, 1, 0, 1, 0, 0, 0]},
                  '2': { 'points': 20300,
                        'riichi': False,
                        'discards': [18, 17, 11, 16, 24, 25, 28, 23, 33, 32, 20, 7, 30, 17],
```

```

23     'melds': [{ 'type': 5, 'tiles': [124, 126, 127, 125], 'who': [2,
24                 2, 1, 2]}],
25     { 'type': 2, 'tiles': [5, 13, 10, -1], 'who': [2, 2, 1, -1]}],
26     'tsumo_giri': [0, 0, 0, 1, 0, 0, 0, 1, 0, 1, 1, 0, 1, 1]},
27     '3': { 'points': 19000,
28           'riichi': False,
29           'discards': [28, 29, 30, 18, 0, 11, 9, 27, 27, 0, 29, 26, 24, 25
30                       ],
31           'melds': [],
32           'tsumo_giri': [0, 0, 0, 0, 0, 0, 1, 0, 0, 1, 1, 0, 0, 1]}],
33     'valid_actions': [{ 'type': 1, 'tiles': [1], 'who': [0, -1, -1, -1]
34                       },
35                       { 'type': 1, 'tiles': [1], 'who': [0, -1, -1, -1]}],
36                       { 'type': 1, 'tiles': [2], 'who': [0, -1, -1, -1]}],
37                       { 'type': 1, 'tiles': [3], 'who': [0, -1, -1, -1]}],
38                       { 'type': 1, 'tiles': [3], 'who': [0, -1, -1, -1]}],
39                       { 'type': 1, 'tiles': [12], 'who': [0, -1, -1, -1]}],
40                       { 'type': 1, 'tiles': [13], 'who': [0, -1, -1, -1]}],
41                       { 'type': 1, 'tiles': [18], 'who': [0, -1, -1, -1]}],
42                       { 'type': 1, 'tiles': [19], 'who': [0, -1, -1, -1]}],
43     'action_idx': 7}
44 }

```

Donde podemos obtener las siguientes informaciones:

Cuadro 7: Campos de un Estado de Mahjong

Campo	Tipo	Descripción
round_wind	int	Viento de la ronda (0-3)
num_honba	int	Número de honba (0-4)
num_riichi	int	Número de riichi's (0-3)
player_wind	int	Viento del jugador (0-3)
position	int	Posición del jugador (0-3)
remain_tiles	int	Número de fichas restantes en el muro (0-70)
dora_indicators	list[int]	Lista de indicadores de dora (0-33)
hand_tiles	list[int]	Lista de fichas en la mano del jugador (0-33)
players	dict[int, dict]	Información de los jugadores, incluyendo puntos, riichi, descartes, melds y tsumo_giri.
valid_actions	list[dict]	Lista de acciones válidas para el jugador actual.
action_idx	int	Índice de la acción tomada por el jugador actual.

De estos datos, podemos aún extraer los campos de players:

Cuadro 8: Campos de un Jugador en un Estado de Mahjong

Campo	Tipo	Descripción
points	int	Puntos del jugador (0-50000)
riichi	bool	Indica si el jugador ha declarado riichi (True/False)
discards	list[int]	Lista de fichas descartadas por el jugador (0-33)
melds	list[dict]	Lista de melds del jugador, cada uno con tipo, fichas y quién contribuyó.
tsumo_giri	list[int]	Lista indicando si cada ficha fue descartada inmediatamente después de ser robada (1) o no (0).

Campos de melds:

Cuadro 9: Campos de un Meld en un Estado de Mahjong

Campo	Tipo	Descripción
type	int	Tipo de meld (2: Pon, 3: Chii, 4: DaiMinKan, 5: ShouMinKan, 6: AnKan)
tiles	list[int]	Lista de fichas que componen el meld (0-33)
who	list[int]	Lista indicando quién contribuyó a cada ficha del meld (-1 para fichas propias, 0-3 para otros jugadores)

Y, por fin, campos de valid_actions:

Cuadro 10: Campos de una Acción Válida en un Estado de Mahjong

Campo	Tipo	Descripción
type	int	Tipo de acción (0: Skip, 1: Discard, 2: Chi, 3: Pon, 4: DaiMinKan, 5: ShouMinKan, 6: AnKan, 7: Riichi)
tiles	list[int]	Lista de fichas involucradas en la acción (0-33)
who	list[int]	Lista indicando quién realiza la acción y quién contribuye a ella (-1 para el jugador actual, 0-3 para otros jugadores)

7 Arquitectura del Modelo

Como explicado anteriormente en la Subseccion 3.3, nuestro modelo utiliza una arquitectura de Transformers al estilo BERT, ver Figura 3.

A continuación, podemos ver como funciona la Aquitectura del modelo de este proyecto:

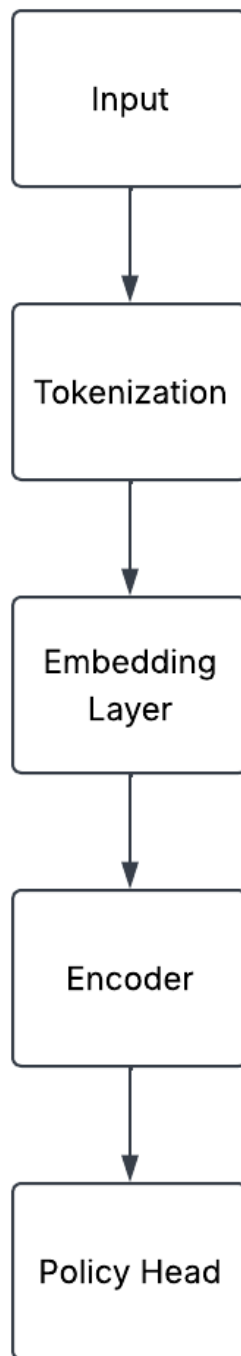


Figura 7: Arquitectura del Modelo Transformer

Esta es una arquitectura simple de un Transformer Encoder, el input es un estado de Mahjong como vimos anteriormente, este estado pasa por la tokenización, donde cada categoría de información del estado se convierte en un token semántico, y cada token es representado por un embedding vectorial. Luego, estos embeddings son procesados por el Transformer Encoder, que captura las relaciones entre los tokens y genera una representación contextualizada del estado. Finalmente, la salida del Transformer se pasa a través de una capa de clasificación para predecir la acción de descarte más adecuada.

7.1 Tokenization

En primer lugar, necesitamos definir las categorías de información de un estado:

Categoría	Ejemplos	Ordenado	Público	Dinámico
Piezas de mano	5m, este	No	No	Sí
Descartes	descarte 9m	Sí	Sí	Sí
Melds	pon/chii/kan	Parcialmente	Sí	Sí
Acciones candidatas	descartar 5m	No	N/A	Sí
Contexto	Viento de la Ronda	No	Sí	Lento
Estatus de los jugadores	riichi, puntos	No	Sí	Sí

Cuadro 11: Categorías de Informaciones

Un estado de Mahjong será representado como una colección de tokens semánticos. Cada token representará una categoría diferente de información de juego y podrá contener información de metadatos del juego.

A continuación veremos para cada categoría, cuáles son los campos que tendrá el token.

7.1.1 Tokens de Mano

Piezas de la mano representarán la identidad de la pieza, una distinción de duplicados (cuál copia de la pieza es), y un indicador de si es dora rojo.

Campo	Objetivo
token_type	identifica el token HAND
tile_type	pieza de Mahjong
copy_index	diferencia copias de piezas
is_red	soporte de dora rojo

Cuadro 12: Campos de Token de Mano

7.1.2 Tokens de Descarte

Los tokens de descarte representan eventos de descartes que son públicos y ordenados, los campos de un token de descarte son los siguientes:

Campo	Objetivo
token_type	identifica el token DISCARD
player	jugador que ha descartado la pieza
tile_type	pieza de Mahjong
copy_index	diferencia copias de piezas
is_red	soporte de dora rojo
is_tsumogiri	si la pieza descartada fue una pieza inmediatamente comprada
is_riichi_declaration	si al descartar esa pieza se declara riichi
global_order	orden cronológico del descarte en la ronda
player_discard_order	índice de descarte para el jugador

Cuadro 13: Campos de Token de Descarte

Campo	Objetivo
token_type	identifica el token MELD
player	jugador que posee el meld
meld_type	chii / pon / kan
tile_types	piezas que componen el meld
copy_indices	distincion de duplicados
red_flags	soporte de dora rojo
source_players	jugadores que contribuyeron para cada pieza
global_order	orden cronologica de llamada

Cuadro 14: Campos de Token de Melds

7.1.3 Tokens de Melds

Los Tokens de Melds representan eventos de melds que son públicos y ordenados, los campos de un token de meld son los siguientes:

Message sent Undo 1 of 4,365

7.1.4 Tokens de Acciones

Los Tokens de Acciones representan eventos de acciones candidatas que son públicos y ordenados, los campos de un token de acción son los siguientes:

Campo	Objetivo
token_type	identifica el token ACTION
acting_player	jugador que realiza la acción
action_index	índice dentro del conjunto de candidatos legales
action_type	descartar / riichi / pon / etc
tile_types	piezas involucradas en la acción
copy_indices	distinción de duplicados
red_flags	soporte de dora rojo
source_players	fuelle de las piezas llamadas
global_order	orden cronológico de la acción

Cuadro 15: Campos de Token de Acciones

7.1.5 Tokens de Estado de Juego

Los Tokens de Estado de Juego representan información contextual del juego que es pública y no ordenada, los campos de un token de estado de juego son los siguientes:

Campo	Objetivo
token_type	identifica el token GAME_STATE
round_wind	ronda Este/Sur
honba_count	honba actual
riichi_sticks	depósitos de riichi
remaining_tiles	piezas restantes en la pared
dora_indicators	indicadores de dora actuales

Cuadro 16: Campos de Token de Estado de Juego

7.1.6 Tokens de Estado de Jugadores

Los Tokens de Estado de Jugadores representan información contextual de cada jugador que es pública y no ordenada, los campos de un token de estado de jugadores son los siguientes:

Campo	Objetivo
token_type	identifica el token PLAYER_STATE
player	identidad del jugador
seat_wind	este/sur/oeste/norte
score	puntos actuales
riichi_status	si el jugador ha declarado riichi
placement	posición actual
open_hand	si el jugador tiene mano abierta

Cuadro 17: Campos de Token de Estado de Jugadores

7.2 Diseño del Embedding

La estructura general del embedding es la siguiente:

$$\text{Embedding}(\text{token}) = \text{token_type_embedding} + \text{semantic_embedding} + \text{metadata_embedding} + \text{positional_embedding}$$

Tipos diferentes de tokens utilizarán diferentes embeddings a depender de su estructura semántica.

Cada categoría de token tendrá un embedding dedicado de tipo de token, que podrá ser uno de los siguientes: HAND, DISCARD, MELD, ACTION, GAME_STATE o PLAYER_STATE.

A continuación, veremos la estructura de embedding para cada token semántico.

7.2.1 Embedding de Tokens de Piezas

Este Embedding codifica la identidad de la pieza, por ejemplo: 1m, 5p, east, red_dragon. Los embeddings de piezas son compartidos entre los tokens de mano, descarte, melds y de acciones.

7.2.2 Embedding del índice de copia

Instancias de piezas duplicadas (por ejemplo, dos 5m) se distinguen mediante un índice de copia. Este embedding permite al modelo diferenciar entre estas instancias.

7.2.3 Embedding de Jugador

Este embedding codifica la identidad del jugador, por ejemplo, jugador 0, jugador 1, etc. Los embeddings de jugadores son compartidos entre los tokens de descarte, melds, acciones y estado de jugadores.

7.2.4 Embeddings Posicionales

Los embeddings posicionales preservan el orden temporal de los eventos de juego, como descartes, momento en que se realizaron melds y extensiones de trayectorias de acciones. Existen dos tipos de esquemas posicionales: orden cronologica global y orden de descarte local de jugador.

7.2.5 Embeddings de Metadatos

Los embeddings de metadatos codifican información adicional relevante para cada token, como indicadores de dora, estado de riichi, y otros atributos contextuales. Estos embeddings permiten al modelo capturar relaciones complejas entre los tokens y el contexto del juego.

7.2.6 Embedding de Token de Mano

Los tokens de mano representan piezas escondidas del jugador actual y es definida por la siguiente estructura de embedding:

$$E_{\text{hand}} = E_{\text{token_type}} + E_{\text{tile}} + E_{\text{copy}} + E_{\text{red}}$$

Dónde:

- $E_{\text{token_type}}$ identifica el token como una entidad de tipo HAND,
- E_{tile} representa la identidad semántica de la pieza,
- E_{copy} distingue entre instancias duplicadas de piezas,
- E_{red} indica si la pieza es un dora rojo.

7.2.7 Embedding de Token de Descarte

Los tokens de descarte representan eventos de descartes que son públicos y ordenados, y su embedding es definido por la siguiente estructura:

$$\begin{aligned} E_{\text{discard}} = & E_{\text{token_type}} + E_{\text{tile}} + E_{\text{copy}} + \\ & E_{\text{red}} + E_{\text{player}} + E_{\text{tsumogiri}} + \\ & E_{\text{riichi_declaration}} + E_{\text{global_position}} + E_{\text{local_position}} \end{aligned} \quad (1)$$

Dónde:

- E_token_type identifica el token como una entidad de tipo DISCARD,
- E_tile representa la identidad semántica de la pieza,
- E_copy distingue entre instancias duplicadas de piezas,
- E_red indica si la pieza es un dora rojo,
- E_player identifica al jugador que ha descartado la pieza,
- E_tsumogiri indica si la pieza descartada fué una pieza inmediatamente comprada,
- E_richi_declaration indica si al descartar esa pieza se declara riichi,
- E_global_position representa el orden cronológico global del descarte,
- E_local_position representa el orden de descarte relativo al jugador.

7.2.8 Embedding de Token de Meld

Tokens de Melds representan llamadas hechas durante la partida, y su embedding es definido por la siguiente estructura:

$$\begin{aligned} E_{\text{meld}} = & E_{\text{token_type}} + E_{\text{player}} + E_{\text{meld_type}} + \\ & E_{\text{tile_set}} + E_{\text{copy_set}} + E_{\text{red_set}} + \\ & E_{\text{source_players}} + E_{\text{global_position}} \end{aligned} \quad (2)$$

Dónde:

- E_token_type identifica el token como una entidad de tipo MELD,
- E_player identifica al jugador que posee el meld,
- E_meld_type representa el tipo de meld (chi, pon, kan, etc),
- E_tile_set codifica las piezas que componen el meld,
- E_copy_set distingue entre instancias duplicadas de piezas,
- E_red_set codifica la información de dora rojo,
- E_source_players identifica los jugadores que contribuyeron a cada pieza del meld,
- E_global_position representa el orden cronológico de la llamada.

7.2.9 Embedding de Token de Acción

Tokens de Acciones representan eventos de acciones candidatas que son públicos y ordenados, y su embedding es definido por la siguiente estructura:

$$\begin{aligned} E_{\text{action}} = & E_{\text{token_type}} + E_{\text{action_type}} + \\ & E_{\text{tile_set}} + E_{\text{copy_set}} + E_{\text{red_set}} + \\ & E_{\text{source_players}} + E_{\text{global_position}} \end{aligned} \quad (3)$$

Dónde:

- $E_{\text{token_type}}$ identifica el token como una entidad de tipo ACTION,
- $E_{\text{action_type}}$ representa descarte, riichi, chi, pon, kan, etc,
- $E_{\text{tile_set}}$ representa las piezas involucradas en la acción,
- $E_{\text{copy_set}}$ distingue entre instancias duplicadas de piezas,
- $E_{\text{red_set}}$ codifica la información de dora rojo,
- $E_{\text{source_players}}$ identifica las fuentes de las piezas para llamadas,
- $E_{\text{global_position}}$ representa el timing de la acción.

7.2.10 Embedding de Token de Estado de Juego

Tokens de Estado de Juego representan información contextual del juego que es pública y no ordenada, y su embedding es definido por la siguiente estructura:

$$\begin{aligned} E_{\text{game}} = & E_{\text{token_type}} + E_{\text{round}} + E_{\text{honba}} + \\ & E_{\text{riichi_sticks}} + E_{\text{remaining_tiles}} + E_{\text{dora_set}} \end{aligned} \quad (4)$$

Dónde:

- $E_{\text{token_type}}$ identifica el token como una entidad de tipo GAME_STATE,
- E_{round} codifica el viento de la ronda (este/sur),
- E_{honba} representa el número de honba actuales,
- $E_{\text{riichi_sticks}}$ codifica la cantidad de depósitos de riichi,
- $E_{\text{remaining_tiles}}$ representa las piezas restantes en la pared,
- $E_{\text{dora_set}}$ codifica los indicadores de dora actuales.

7.2.11 Embedding de Token de Estado de Jugador

El Token de Estado de Jugador representa información contextual persistente asociada con cada jugador, y su embedding es definido por la siguiente estructura:

$$E_{\text{player_state}} = E_{\text{token_type}} + E_{\text{player}} + E_{\text{seat}} + E_{\text{score}} + E_{\text{riichi_status}} + E_{\text{open_hand}} \quad (5)$$

Dónde:

- $E_{\text{token_type}}$ identifica el token como una entidad de tipo `PLAYER_STATE`,
- E_{player} codifica la identidad del jugador,
- E_{seat} representa el viento de asiento del jugador (este/sur/oeste/norte),
- E_{score} codifica los puntos actuales del jugador,
- $E_{\text{riichi_status}}$ indica si el jugador ha declarado riichi,
- $E_{\text{open_hand}}$ indica si el jugador tiene una mano abierta.

Glosario de términos de Mahjong

Pinzu (筒子)

Named as each tile consists of a number of circles.

Referencias

- [1] D. Chiba, *Riichi Book 1: The Ultimate Guide to the Japanese Game Taking the World by Storm*. Lulu.com, 2015, ISBN: 978-1329626478.
- [2] T. Developers, *Tjong AI Mahjong Platform*, <https://github.com/tjong>, Open-source Riichi Mahjong AI platform, 2024.
- [3] J. Devlin, M. Chang, K. Lee y K. Toutanova, “BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding”, *CoRR*, vol. abs/1810.04805, 2018. arXiv: 1810.04805. dirección: <http://arxiv.org/abs/1810.04805>
- [4] A. Hussein et al., “Imitation Learning: A Survey of Learning Methods”, *ACM Computing Surveys*, 2017.
- [5] Japanese Mahjong Wiki. “Rules overview”, visitado 9 de jun. de 2026. dirección: https://riichi.wiki/Rules_overview
- [6] J. Li et al., “Suphx: Mastering Mahjong with Deep Reinforcement Learning”, *arXiv preprint arXiv:2003.13590*, 2020.
- [7] A. Tam, *Encoders and Decoders in Transformer Models*, <https://machinelearningmastery.com/encoders-and-decoders-in-transformer-models/>, Accessed: 2026-06-26, sep. de 2025.
- [8] A. Vaswani et al., “Attention Is All You Need”, *NeurIPS*, 2017.